

Projet de programmation BD et Web

Pagès Stanislas, Jorel Raphaël

Table des matières

I	Introduction	3
1	Pour commencer	3
2	Le sujet	3
II	Modèle entités - relations	4
III	Justification du modèle	5
3	Entités	5
3.1	Formations et Entreprises	5
3.2	Enseignants	5
3.3	Tuteurs	5
3.4	Stages	5
3.5	Etudiants	6
4	Relations	6
4.1	Propose, Désigne et Encadre	6
4.2	Concerne, Etudie, Enseigne et Tutelle	6
4.3	Demande (accordé), Contrôle	6
IV	Programmation du site	7
5	Eléments généraux	7
5.1	Membres	7
5.2	Modèle de programmation	7
5.3	Sessions utilisateur	7
6	Base de données	7
7	Front et back controllers	8

8	Partie visible	8
8.1	Informations	8
8.2	Enseignant responsable	8
8.3	Enseignant tuteur	9
8.4	Entreprise	9
8.5	Tuteur d'entreprise	9
8.6	Elève	9
9	Sécurité	9
10	Améliorations	10
V	Conclusion	11
VI	Annexes	12
11	Création de la base de données	12

Première partie

Introduction

1 Pour commencer

Il nous a été demandé de réaliser un site web qui gère des stages d'entreprises pour l'Université de Perpignan Via Domitia. Des entreprises proposent des stages sur ce site, validés par des professeurs administrateurs, qui peuvent ensuite être demandés par les élèves. Les stages leur sont accordés sur critères que les professeurs administrateurs déterminent eux-même (niveau des stages adaptés, par exemple).

2 Le sujet

Ce projet se découpe principalement en deux parties, la première concerne la conception de la base de données utilisée, la seconde se concentre sur la mise en forme de ces données, ainsi que l'interaction des utilisateurs avec la base de données.

Nous avons fait la description de la finalité du site ci-dessus, ce à quoi il nous faut rajouter quelques précisions. Tout d'abord, la nature des personnes (ou groupes) présentes sur le site, à savoir :

- des entreprises,
- des tuteurs d'entreprises,
- des enseignants responsables,
- des enseignants tuteurs,
- des étudiants.

Les entreprises proposent des stages, et leur affectent des tuteurs (un par stage).

Les tuteurs d'entreprises peuvent assister aux soutenances de stages effectuées à la fin de ces derniers.

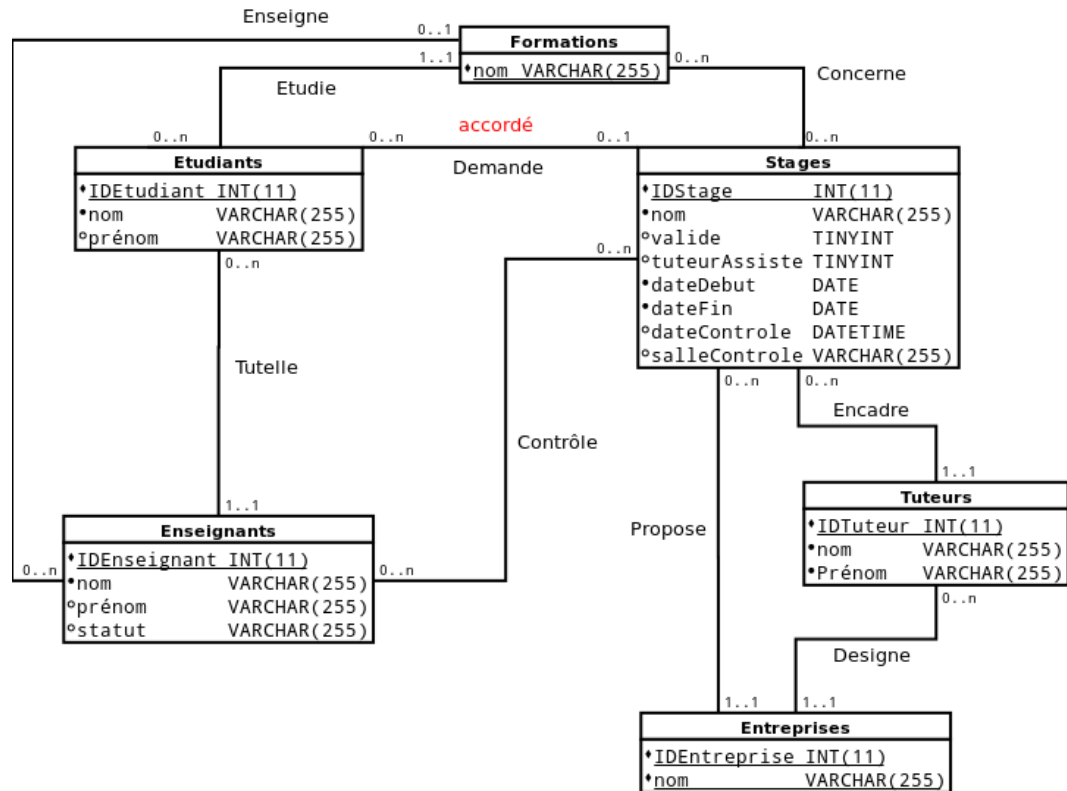
Les enseignants responsables (administrateurs) ont tous les droits sur les listes d'entreprises, d'enseignants et d'étudiants, ce sont eux les gérants (d'où leur nom). Ils valident les stages des entreprises en leur affectant des formations requises pour être effectués par les étudiants, et les accordent ensuite aux élèves voulant les faire. Ils choisissent également des jurys pour contrôler les soutenances à la fin des stages. Ces derniers sont composés d'enseignants, ils ont une date et une salle pour l'examen.

Les enseignants tuteurs sont affectés à des élèves, dont ils peuvent consulter les stages de ces derniers, ainsi que les entreprises qui les proposent. Enfin, **Les étudiants** choisissent les stages précédemment validés.

Nous avons donc dû commencer par le modèle de la base de données, qui nous est donné par la modélisation entités - relations ci après.

Deuxième partie

Modèle entités - relations



Ceci est donc la modélisation graphique. Maintenant, il est important d'expliquer pourquoi nous avons choisi un tel modèle.

Troisième partie

Justification du modèle

3 Entités

Ce modèle ne comporte pas beaucoup d'entités, aussi allons-nous les expliquer une par une. Nous allons commencer avec les plus simples, celle qui comportent le moins de champs, puis nous remonterons vers celles qui dépendent des précédentes. Pour finir, nous parlerons des différentes relations.

3.1 Formations et Entreprises

Les formations ne comportent qu'un nom, qui est également en clé primaire, ainsi une formation est unique. Il en est de même pour les entreprises, car leur nom est également en clé primaire avec leur identifiant.

La raison de l'utilisation d'un identifiant pour les entreprises est uniquement dû au site lui-même, car chaque utilisateur est reconnu par son identifiant et son statut. Le champ *IDEntreprise* aurait pu être enlevé si l'on avait procédé autrement.

Il faut remarquer également que ce sont deux entités qui ne dépendent d'aucune autre.

3.2 Enseignants

Un professeur a un identifiant, nom, un prénom, et un statut, pour différencier les professeurs responsables, et les tuteurs. Un professeur tuteur peut très bien n'avoir aucun étudiant sous tutelle. Nous avons permis également que les professeurs n'aient pas forcément de formation, car par exemple, un professeur responsable peut être juste administrateur du site et rien de plus. Cette entité ne dépend d'aucune autre, comme les deux précédentes.

3.3 Tuteurs

Un tuteur d'entreprise a également un identifiant, un nom et un prénom. Il est rattaché à une entreprise, il en dépend donc. Il peut être affecté à un stage par son entreprise.

3.4 Stages

La plus grosse entité de la base de données, un stage a un identifiant, un nom (ou intitulé), une date de début et une autre de fin, une date et une salle d'examen. Il est aussi composé du champ *valide*, qui, comme son nom l'indique, permet de savoir s'il est valide pour être choisi par les étudiants. C'est le rôle de l'administrateur de gérer cela. Le champ *tuteurAssiste* permet de savoir si le tuteur du stage souhaite être présent lors de la soutenance de ce dernier. Il dépend donc d'un tuteur et d'une entreprise.

3.5 Etudiants

Pour terminer, l'entité qui donne un sens à tout le reste.

Un étudiant a un identifiant, un nom et un prénom. Il peut choisir un stage valide, il a un enseignant tuteur et une formation, obligatoirement.

4 Relations

Nous détaillerons maintenant les relations. La plupart sont très explicites de par leur nom, cependant nous allons les présenter pour éviter toutes ambiguïtés.

4.1 Propose, Désigne et Encadre

Ces trois relations concernent les entreprises et leur tuteurs. L'entreprise **propose** zéro ou plusieurs stages, tandis qu'un stage ne peut être proposé que par une seule entreprise (pas de stages partagés), il en est de même des tuteurs d'entreprises, qui sont **désignés** par ces dernières. Un tuteur peut **encadrer** zéro ou plusieurs stages, tandis qu'un stage ne peut avoir qu'un tuteur.

4.2 Concerne, Etudie, Enseigne et Tutelle

Ces relations-là se rapporte à l'université. Un stage peut **concerner** zéro ou plusieurs formations, l'inverse est également possible. Un étudiant **étudie** dans une formation, mais une formation peut être suivie par plusieurs élèves, ou aucun. Un professeur peut **enseigner** dans zéro ou une formation. Un professeur sans formation est assez difficilement concevable, mais cela ajoute de la souplesse au site (nous l'avons mentionné plus haut). Par contre, à l'instar des élèves, plusieurs professeurs peuvent enseigner la même formation, ou aucun. Et enfin, un élève est placé sous **tutelle** d'un professeur habilité à cela, il n'en a qu'un, tandis qu'un professeur tuteur peut avoir plusieurs élèves sous tutelle, ou aucun.

4.3 Demande (accordé), Contrôle

Pour finir, les relations qui lient l'université aux entreprises. Plusieurs élèves peuvent **demander** le même stage. par contre, il ne peuvent en demander qu'un seul, qui leur sera **accordé** ou non. Evidemment, si plusieurs élèves demandent le même stage, un seul peut l'avoir, et les autres verront automatiquement un refus pour leur demande. Et évidemment, un élève peut ne pas avoir envie de suivre un stage. Des enseignants (responsables ou tuteurs) sont désignés pour **contrôler** les élèves en leur faisant passer des soutenances de stages quand ces derniers sont finis. Il peut n'y avoir aucun enseignant qui participe à la soutenance d'un stage si celui-ci n'est pas encore validé.

Maintenant que le modèle est défini et justifié, il est temps d'expliquer la programmation du site.

Quatrième partie

Programmation du site

5 Eléments généraux

5.1 Membres

Il y avait beaucoup de possibilités devant nous pour programmer ce site, comme dans tout projet, mais ici un problème important apparaît tout de suite : la présence de membres dans le site. Comment savoir si la personne est étudiante, ou professeur, ou une entreprise ? Voilà qui justifie l'ajout d'une nouvelle table, mais ne répondant pas au problème posé, nous ne l'avons pas mis dans le schéma plus haut. Cette table est décrite, comme toutes les autres en 11. Celle-ci présente un login, un mot de passe, le statut de la personne (étudiant, professeur, entreprise, tuteur), et son identifiant dans la table correspondante. Ainsi le site redirige automatiquement vers les pages accessibles à chaque catégorie de personnes.

5.2 Modèle de programmation

Problème important également, quels choix de programmation ? Toute personne censée sait qu'il faut avoir une certaine rigueur en programmation, sinon le résultat est tout bonnement ingérable. Nous nous sommes donc basés sur l'architecture Model - View - Controller.

Evidemment, nous l'avons utilisé comme principe, mais il n'est pas vraiment respecté. Disons que cela nous permettait de nous en tenir à une manière d'organiser le site.

5.3 Sessions utilisateur

Encore quelque chose de non-négligeable, la session utilisateur. Nous avons donc choisi d'utiliser les sessions fournies par PHP, plutôt que de passer par des cookies. Grâce à celles-ci, nous pouvons retenir les informations essentielles concernant l'utilisateur en cours.

6 Base de données

Celle-ci est gérée par une classe, la classe Database qui ne comporte qu'un attribut, le lien avec la base de données. Nous avons utilisé le modèle objet de l'interface PHP avec MySQL (PDO), car celui-ci permet les requêtes préparées, une sécurité en plus pour le site (bien qu'aucun hacker ne devrait un jour vouloir pirater ce dernier). Cela permet dans le reste du code de n'instancier qu'un seul objet, qui sera le lien entre le script et la base de données. Cette classe contient donc beaucoup de méthodes, toutes relatives à l'interaction avec la base de données, la plupart sont des requêtes de type interrogation, quelques unes de type insertion et modification, et un peu moins de type suppression.

7 Front et back controllers

Le *front controller* est ici le fichier qui gère toutes les redirections de connexion. Il permet de se connecter/déconnecter et redirige vers le *back controller* principal. Ce dernier gère les redirections vers d'autres *back controllers* plus spécifiques à chaque type de membres, pour gérer leurs actions et les pages présentées. Dans le cas où les actions d'un membre sont très limitées, il se charge lui-même de les traiter. Les autres *back controllers* fonctionnent de la même manière, ils traitent les informations données par l'utilisateur et affichent les pages en fonction des choix de l'utilisateur.

Le système de contrôleurs est inhérent à l'architecture MVC, donc nous avons suivi ce principe car il aide vraiment à départager le code de façon claire. Il est également important de présenter des pages que l'utilisateur verra. Voyons donc maintenant cette partie.

8 Partie visible

C'est grâce à cette partie que l'utilisateur voit et peut donner des informations au site, bien que celles-ci soient traitées ailleurs. Nous allons maintenant expliquer un peu plus chaque interface, pour apporter quelques précisions.

8.1 Informations

Cette fonctionnalité permet à l'utilisateur d'avoir des informations plus précises sur les éléments de la base de données. Quand c'est possible, les entités sont cliquables, cela redirige vers cette page d'informations. Cette possibilité est vraiment très utile car elle permet de ne pas encombrer les autres pages d'informations, et de les regrouper sur une seule. Par exemple, si l'administrateur doit accorder un stage à un élève, il clique sur l'élève, voit sa formation, il clique ensuite sur le stage, et voit les formations le concernant et ainsi, il décide si l'élève peut le suivre, plutôt que de surcharger la page de validation avec toutes les informations du stage, et celles de l'élève, surtout s'il y a plusieurs élèves qui demandent des stages.

8.2 Enseignant responsable

L'administrateur peut voir toutes les listes, ajouter, modifier et supprimer des comptes sur le site. Un administrateur ne peut pas ajouter un tuteur d'entreprise, ce qui paraît assez logique. Il peut par contre ajouter des entreprises, des élèves, des professeurs et des formations. Il peut également modifier les informations des entreprises, des enseignants et des étudiants, ainsi que les supprimer. Un professeur tuteur avec des élèves sous sa tutelle ne pourra être supprimé qu'une fois que ceux-ci le seront. Supprimer une entreprise revient à supprimer tous ces stages, tuteurs du site.

Lorsque l'administrateur valide un stage, il a accès à toutes les informations utiles du stage (début du stage, fin, intitulé) afin de pouvoir donner une date pour la soutenance, ainsi qu'un jury où il peut désigner tous les professeurs qu'il

veut. A noter qu'aucun descriptif de stage n'apparaît dans un stage, car cela n'est qu'un confort ajouté à l'utilisateur, la date de fin de stage, par exemple, est plus importante pour pouvoir déterminer la date de soutenance. De toute manière, l'intitulé d'un stage suffit bien souvent à comprendre en quoi il consiste. La dernière action possible de l'administrateur est d'autoriser les stages aux élèves qui demandent. La page affiche juste les élèves et les stages, avec de quoi valider ou pas.

Pour finir, l'administrateur peut aussi consulter les listes des élèves, des entreprises, etc... La page qui concerne cela est une entorse à l'architecture MVC, car elle est censée n'être qu'une page de rendu, or elle fait appel à quelques fonctions de la base de données. Mais elle permet de gérer les listes d'utilisateurs du site et des formations.

8.3 Enseignant tuteur

L'enseignant tuteur a un rôle très limité, il n'a aucune action à sa disposition, il peut juste consulter la liste des élèves sous sa tutelle ainsi que les stages auxquels ces derniers participent et enfin les entreprises qui proposent ces stages. Cela se fait de manière très simple grâce aux fiches d'informations décrites ci-dessus.

8.4 Entreprise

L'entreprise peut déclarer des tuteurs, ainsi que soumettre des stages, en indiquant les informations nécessaires. La date de fin de stage doit être cohérente avec la date de début, c'est la seule contrainte que nous imposons et qui est assez logique. Elle peut également supprimer un stage refusé ou qui n'a pas encore été validé.

8.5 Tuteur d'entreprise

Un tuteur d'entreprise est aussi très limité dans ses actions, il ne peut que préciser s'il souhaite participer aux soutenance de stages des élèves qui suivent ceux auxquels il est affecté.

8.6 Elève

Les élèves ont le droit de choisir un stage parmi ceux proposés, donc validés et non suivis par d'autres. Il ne peut demander plus d'un stage à la fois, qui lui sera donc refusé ou accordé. S'il est accordé, un petit message s'écrit à côté du stage qu'il a demandé, sinon, un autre lui sera affiché à sa prochaine connexion lui informant du refus qu'a prononcé un administrateur. Ce après quoi, le message ne s'affichera bien évidemment plus.

Maintenant, parlons d'un aspect moins important pour ce site en local : la sécurité.

9 Sécurité

Il serait bien prétentieux de dire que notre site est sécurisé, surtout que cet aspect n'a pas été traité longuement. Nous avons juste essayé de le rendre un

peu moins instable la plupart des fois où les informations transitent par l'URL. Par exemple un élève, s'il est malin et un peu chercheur, peut demander un stage non validé, car son identifiant est envoyé via l'URL. Il suffit donc de faire une vérification (existence + validité) du stage demandé. Il peut aussi demander un stage déjà accordé. Là encore, une vérification s'impose pour ne pas perturber la base de données.

Les entreprises peuvent supprimer des stages. Là encore l'information est facilement modifiable puisqu'elle apparaît encore dans l'URL. Il faut donc faire une vérification et être bien sûr que le stage appartient à l'entreprise.

De même pour les tuteurs d'entreprises, qui peuvent indiquer s'ils veulent participer aux soutenances, nous contrôlons d'abord que les stages les concernent.

En ce qui concerne les administrateurs, aucune vérification n'est faite, car étant les responsables, il est de leur devoir de ne pas faire n'importe quoi. De toute manière, il peuvent tout faire de manière conventionnelle, il n'y a pas de raison qu'ils s'amuse à modifier les URLs. Cela peut juste poser problème s'ils changent les valeurs dans cette dernière et accèdent à des informations n'existant pas (identifiant de stage inexistant par exemple), mais ils tomberont sur des erreurs de script, rien de plus normalement.

D'une manière générale, les vérifications ne sont faites que lorsque l'on modifie les données du site, dans les cas où l'on demande juste des informations, rien n'est sécurisé. Nous partons du principe que si un élève est capable d'accéder aux informations d'un stage non-validé, ou autres actions normalement impossibles, grand bien lui fasse, mais comme cela ne dérange pas l'état interne du système, il n'y a pas de problème.

10 Améliorations

Pour finir, nous exposerons les améliorations à faire pour que le site soit le plus complet possible. En voici une liste non-exhaustive :

- permettre aux utilisateurs de changer de mot de passe,
- permettre de supprimer un stage lorsqu'il sera fini et contrôlé,
- permettre en général aux utilisateurs de revenir sur leurs décisions (validations de stages, participations aux soutenances, demandes de stages, etc...),
- renforcer la sécurité concernant l'accès aux données.

Bien que toutes ces fonctionnalités soient un surplus qui ne répondent pas plus au sujet, si le site devait être mis en ligne, il les lui faudrait bien.

Cinquième partie

Conclusion

Nous avons essayé d'expliquer notre solution le plus clairement et précisément possible, nous espérons donc y être parvenu. Nous savons pertinemment que ce site ne respecte pas le modèle de programmation que nous nous sommes donnés à la base, mais que nous avons utilisé comme ligne directrice. Pour conclure, nous dirons que ce sujet fut un très bon exercice de manipulation de base de données, bien que la plus grosse partie du travail ce soit passée dans le script.

Sixième partie

Annexes

11 Création de la base de données

```
1
2
3 — Tous les droits aux utilisateurs.
4
5 GRANT ALL PRIVILEGES ON project.* TO raphael;
6 GRANT ALL PRIVILEGES ON project.* TO stanislas;
7
8
9 — Table des membres du site.
10
11 CREATE TABLE IF NOT EXISTS 'Members' (
12     'login' VARCHAR(255) NOT NULL,
13     'password' VARCHAR(255) NOT NULL,
14     'status' VARCHAR(255) NOT NULL,
15     'memberID' INT NOT NULL,
16     PRIMARY KEY ('login')
17     ) ENGINE=InnoDB DEFAULT CHARSET=utf8 COLLATE=utf8_bin;
18
19
20 — Table des entreprises.
21
22 CREATE TABLE IF NOT EXISTS 'Companies' (
23     'ID' INT NOT NULL AUTO_INCREMENT,
24     'name' VARCHAR(255) NOT NULL,
25     PRIMARY KEY ('ID', 'name')
26     ) ENGINE=InnoDB DEFAULT CHARSET=utf8 COLLATE=utf8_bin;
27
28
29 — Table des formations.
30
31 CREATE TABLE IF NOT EXISTS 'Formations' (
32     'name' VARCHAR(255) NOT NULL,
33     PRIMARY KEY ('name')
34     ) ENGINE=InnoDB DEFAULT CHARSET=utf8 COLLATE=utf8_bin;
35
36
37 — Table des enseignants (tuteurs ou responsables).
38
39 CREATE TABLE IF NOT EXISTS 'Teachers' (
40     'ID' INT NOT NULL AUTO_INCREMENT,
41     'lastName' VARCHAR(255) NOT NULL,
42     'firstName' VARCHAR(255) NOT NULL,
43     'status' VARCHAR(255) NOT NULL,
44     'formationName' VARCHAR(255),
45     FOREIGN KEY ('formationName') REFERENCES 'Formations' ('name'),
46     PRIMARY KEY ('ID')
47     ) ENGINE=InnoDB DEFAULT CHARSET=utf8 COLLATE=utf8_bin;
48
49
50 — Tables des tuteurs d'entreprises.
51
52 CREATE TABLE IF NOT EXISTS 'Tutors' (
53     'ID' INT NOT NULL AUTO_INCREMENT,
54     'lastName' VARCHAR(255) NOT NULL,
55     'firstName' VARCHAR(255) NOT NULL,
56     'companyID' INT NOT NULL,
57     FOREIGN KEY ('companyID') REFERENCES 'Companies' ('ID'),
58     PRIMARY KEY ('ID')
59     ) ENGINE=InnoDB DEFAULT CHARSET=utf8 COLLATE=utf8_bin;
60
61
62 — Table des stages.
63
64 CREATE TABLE IF NOT EXISTS 'Internships' (
65     'ID' INT NOT NULL AUTO_INCREMENT,
66     'name' VARCHAR(255) NOT NULL,
67     'tutorAssist' TINYINT,
68     'available' TINYINT,
69     'dateBegin' DATE NOT NULL,
70     'dateEnd' DATE NOT NULL,
71     'dateTest' DATETIME,
72     'roomTest' VARCHAR(255),
73     'companyID' INT NOT NULL,
74     'tutorID' INT NOT NULL,
75     FOREIGN KEY ('companyID') REFERENCES 'Companies' ('ID'),
76     FOREIGN KEY ('tutorID') REFERENCES 'Tutors' ('ID'),
77     PRIMARY KEY ('ID')
78     ) ENGINE=InnoDB DEFAULT CHARSET=utf8 COLLATE=utf8_bin;
79
80
81 — Table répertoriant les professeurs qui contrôlent un stage.
```

```

82 CREATE TABLE IF NOT EXISTS 'TeachersToInternships' (
83     'teacherID' INT NOT NULL,
84     'internshipID' INT NOT NULL,
85     FOREIGN KEY ('teacherID') REFERENCES 'Teachers' (('ID'),
86     FOREIGN KEY ('internshipID') REFERENCES 'Internships' (('ID'),
87     PRIMARY KEY ('teacherID', 'internshipID')
88 ) ENGINE=InnoDB DEFAULT CHARSET=utf8 COLLATE=utf8_bin;
89
90
91 — Table répertoriant les formations nécessaires à un stage.
92
93 CREATE TABLE IF NOT EXISTS 'InternshipsToFormations' (
94     'internshipID' INT NOT NULL,
95     'formationName' VARCHAR(255) NOT NULL,
96     FOREIGN KEY ('internshipID') REFERENCES 'Internships' (('ID'),
97     FOREIGN KEY ('formationName') REFERENCES 'Formations' (('name'),
98     PRIMARY KEY ('internshipID', 'formationName')
99 ) ENGINE=InnoDB DEFAULT CHARSET=utf8 COLLATE=utf8_bin;
100
101
102 — Table des étudiants.
103
104 CREATE TABLE IF NOT EXISTS 'Students' (
105     'ID' INT NOT NULL AUTO_INCREMENT,
106     'lastName' VARCHAR(255) NOT NULL,
107     'firstName' VARCHAR(255) NOT NULL,
108     'formationName' VARCHAR(255) NOT NULL,
109     'teacherID' INT NOT NULL,
110     'internshipID' INT,
111     'internshipGranted' TINYINT,
112     FOREIGN KEY ('teacherID') REFERENCES 'Teachers' (('ID'),
113     FOREIGN KEY ('internshipID') REFERENCES 'Internships' (('ID'),
114     PRIMARY KEY ('ID')
115 ) ENGINE=InnoDB DEFAULT CHARSET=utf8 COLLATE=utf8_bin;
116
117
118 INSERT INTO Teachers VALUES(1, 'Jorel', 'Raphaël', 'admin', NULL);
119 INSERT INTO Teachers VALUES(2, 'Pagès', 'Stanislas', 'admin', NULL);
120 INSERT INTO Members VALUES('raphael_m4a1', SHA1('truc'), 'admin', 1);
121 INSERT INTO Members VALUES('stan', SHA1('motdepasse'), 'admin', 2);
122

```